

Laboratorio evolutivo 03

Datos y almacenamiento de Digital Café Luna en AWS

Cloud Computing Posgrado · Semana 3 · Cuenta AWS estándar

Duración	Modalidad	Entorno	Carácter
90 minutos	Equipos de 4	Cuenta AWS estándar	Formativo; con evidencia

1. Propósito y resultado

Demostrar que el estado importante puede sobrevivir al ciclo de vida del cómputo y que object, block y file responden a patrones de acceso distintos.

Al finalizar, el equipo habrá protegido versiones de un objeto en Amazon S3 y habrá demostrado un archivo compartido en Amazon EFS: una instancia lo escribe, una segunda lo lee y el dato permanece en el filesystem, no en el ciclo de vida de una instancia concreta.

Objetivos de aprendizaje

- Relacionar el patrón de acceso con la elección entre object, block y file.
- Usar S3 Versioning para conservar dos versiones del mismo objeto y consultar una versión anterior.
- Configurar un lifecycle sencillo y explicar qué problema operativo resuelve.
- Montar un EFS Regional desde cómputo temporal en dos Availability Zones y observar estado compartido.
- Distinguir persistencia de datos de persistencia del cómputo y justificar el trade-off de cada opción.

Alcance de esta iteración

S3 y EFS son hands-on. EBS se usa como contraste arquitectónico; no se crea ningún volumen adicional. La conectividad y los security groups existen solo para habilitar la prueba de EFS. No trabajaremos NAT Gateway, routing avanzado, NACL, IAM/KMS en profundidad, observabilidad, tuning de storage ni DR multi-Region.

Ruta de trabajo

Tiempo	Bloque	Salida
0–10 min	Preflight + conectividad mínima	Baseline S01 y acceso temporal listos
10–18 min	Crear EFS temprano	Filesystem + mount targets aprovisionando
18–40 min	S3: versiones + recuperación	Dos versiones y versión anterior leída
40–48 min	S3 Lifecycle + evidencia	Regla activa y comportamiento explicado
48–62 min	EC2 temporal	Dos clientes en AZ distintas
62–76 min	EFS: write/read compartido	Mismo archivo visible desde A y B
76–82 min	Contraste object/block/file	Decisión y trade-off explicados
82–90 min	Evidencia + cleanup	Baseline S01 restaurado

Arquitectura objetivo

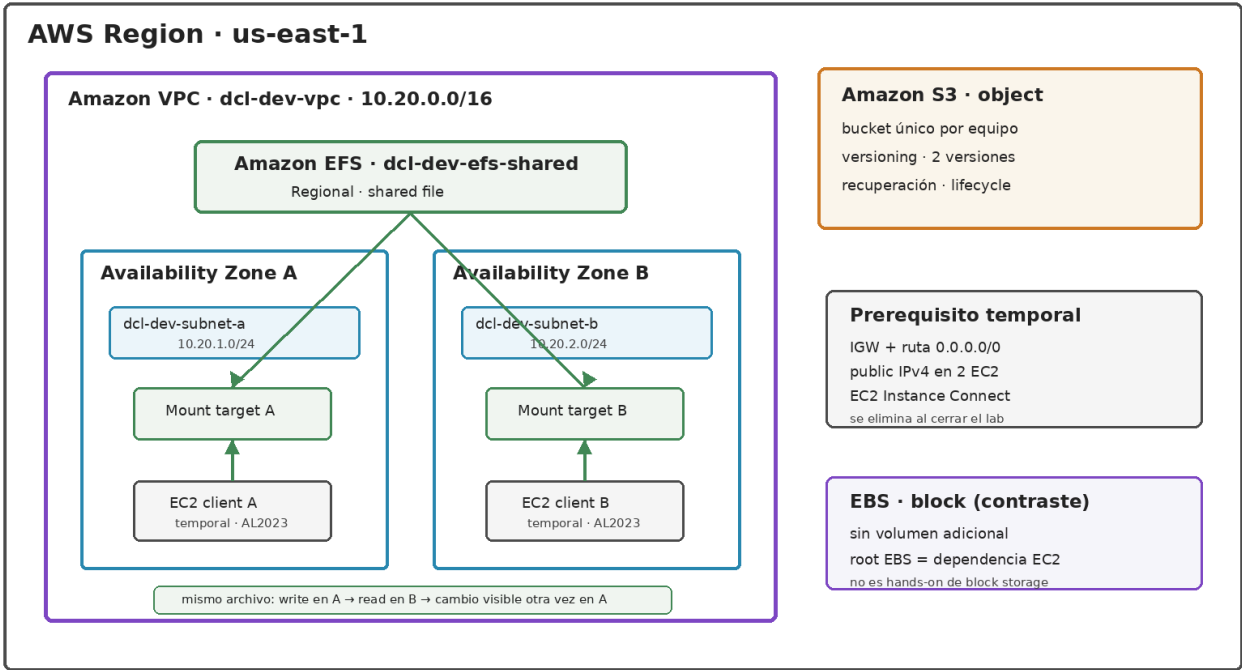


Figura 1. S03 desacopla el estado del cómputo: S3 conserva versiones de objetos y EFS expone un filesystem compartido a dos clientes temporales.

El baseline de S01 permanece intacto. Para EFS se crea conectividad temporal únicamente porque las instancias necesitan acceso a Internet para instalar el cliente NFS y para usar EC2 Instance Connect. El montaje usa la IP del mount target de cada AZ, por lo que no es necesario modificar los atributos DNS de la VPC.

2. Estándar del laboratorio

Región y baseline

Recurso	Nombre / valor
Región	us-east-1
VPC	dcl-dev-vpc · 10.20.0.0/16
Subnet A	dcl-dev-subnet-a · 10.20.1.0/24
Subnet B	dcl-dev-subnet-b · 10.20.2.0/24
Distribución	Dos Availability Zones distintas

Nomenclatura S03

Recurso	Nombre
EFS	dcl-dev-efs-shared
Security group EFS	dcl-dev-sg-efs-s03
Security group EC2	dcl-dev-sg-ec2-s03
Internet Gateway temporal	dcl-dev-igw-s03
Route table temporal	dcl-dev-rt-s03-public
EC2 cliente A	dcl-dev-s03-client-a
EC2 cliente B	dcl-dev-s03-client-b
S3 bucket	dcl-s03-teamXX-<account-id>-<suffix>

El bucket de S3 debe ser globalmente único. Usen minúsculas, el número de equipo, el Account ID y un sufijo corto aleatorio. Apliquen las etiquetas del proyecto cuando el servicio lo permita: `dcl:project=digital-cafe-luna`, `dcl:environment=dev`, `dcl:owner-team=team-XX` y `dcl:managed-by=console`.

3. Preflight técnico y gate de EFS

Qué hacemos: Confirmar cuenta, Región, baseline, permisos y la ruta mínima de acceso que usará el cómputo temporal.

Por qué: EFS requiere mount targets dentro de la VPC y NFS permitido por security groups. Las dos EC2 necesitan acceso temporal a Internet para instalar `nfs-utils` y usar EC2 Instance Connect; no necesitamos NAT Gateway ni cambiar el diseño de red de S01.

1. Inicien sesión en la cuenta AWS y seleccionen `us-east-1`.
2. Confirмен `dcl-dev-vpc`, `dcl-dev-subnet-a` y `dcl-dev-subnet-b` con sus CIDR exactos y AZ distintas.
3. No asuman recursos de S02. Si quedó algún ALB, ASG, IGW, route table o instancia temporal de esa semana, finalicen primero su cleanup y no los reutilicen como dependencia de S03.
4. Confirмен que la identidad puede administrar S3, EFS, EC2, security groups, Internet Gateway y route tables, y que puede usar EC2 Instance Connect. Si una política IAM/SCP bloquea una operación, registren el error y escálenlo; no creen access keys como workaround.
5. Si usarán CloudShell para los comandos de S3, ábralo ahora. Si no está disponible para la identidad, ejecuten las validaciones equivalentes desde la consola.

Validación: Región y baseline coinciden con S01; no existe dependencia activa de S02; la cuenta permite crear los recursos mínimos de S03.

Paso 1 — Preparar conectividad y security groups mínimos

Qué hacemos: Dar acceso temporal a Internet a las dos subnets y definir solo los flujos necesarios para EC2 Instance Connect y NFS.

Por qué: El acceso a Internet es una dependencia operativa para instalar el cliente NFS y entrar al terminal de las instancias. No es el objetivo pedagógico del lab y se elimina al cierre.

6. Creen `dcl-dev-igw-s03` y adjúntenlo a `dcl-dev-vpc`.
7. Creen `dcl-dev-rt-s03-public`, agreguen `0.0.0.0/0` → `dcl-dev-igw-s03` y asóciénla explícitamente a `dcl-dev-subnet-a` y `dcl-dev-subnet-b`.
8. Creen `dcl-dev-sg-ec2-s03`. Inbound: SSH TCP 22 desde la AWS-managed prefix list `com.amazonaws.us-east-1.ec2-instance-connect`. Outbound: conserven la regla predeterminada.
9. Creen `dcl-dev-sg-efs-s03`. Inbound: NFS TCP 2049 con source = `dcl-dev-sg-ec2-s03`. Outbound: conserven la regla predeterminada.

Validación: las dos subnets usan temporalmente la route table S03; el SG de EFS solo recibe NFS desde el SG de las instancias; no existe NAT Gateway ni regla NFS abierta a Internet.

Paso 2 — Crear EFS temprano

Qué hacemos: Crear el filesystem y sus dos mount targets antes de trabajar S3 para aprovechar el tiempo de aprovisionamiento.

Por qué: Un filesystem EFS Regional puede ser accedido por varios clientes; cada AZ desde la que montaremos tendrá un mount target local.

10. En Amazon EFS, creen dcl-dev-efs-shared como filesystem Regional. Mantengan General Purpose y Elastic throughput. Dejen cifrado en reposo habilitado.
11. Deshabiliten automatic backups para este filesystem temporal. No abriremos AWS Backup en esta sesión.
12. En Network, seleccionen dcl-dev-vpc y creen un mount target en dcl-dev-subnet-a y otro en dcl-dev-subnet-b. Asociar dcl-dev-sg-efs-s03 a ambos.
13. Registren el File system ID y, cuando aparezcan, la IP privada de cada mount target. No esperen ociosamente: continúen con S3 mientras los mount targets pasan a Available.

Validación: dcl-dev-efs-shared existe, usa dos mount targets en AZ distintas y ambos están asociados al SG NFS mínimo.

4. S3 hands-on — el objeto conserva su historia

Paso 3 — Crear el bucket y habilitar versioning

Qué hacemos: Crear un bucket del equipo y proteger la historia de un objeto con Versioning.

Por qué: En object storage una nueva escritura con la misma key puede convertirse en una nueva versión en lugar de sobrescribir irreversiblemente el estado anterior.

14. En Amazon S3, creen el bucket único acordado en us-east-1. Mantengan Block all public access habilitado y las opciones de seguridad predeterminadas.
15. Habiliten Bucket Versioning desde la creación o en Properties inmediatamente después.
16. Registren el nombre exacto del bucket en su bitácora y, si usarán CloudShell, asígnenlo a la variable BUCKET.

```
export BUCKET="dcl-s03-teamXX-<account-id>-<suffix>"
aws s3api get-bucket-versioning --bucket "$BUCKET"
```

Validación: Status = Enabled para versioning y el bucket está en la Región de trabajo.

Paso 4 — Crear dos versiones del mismo objeto

Qué hacemos: Subir dos contenidos distintos usando la misma key estado.txt.

Por qué: La key permanece estable para el consumidor, mientras S3 conserva version IDs distintos para cada estado del objeto.

```
printf 'pedido=1001 | estado=CREADO\n' > estado.txt
aws s3 cp estado.txt "s3://${BUCKET}/estado.txt"

printf 'pedido=1001 | estado=PROCESADO\n' > estado.txt
aws s3 cp estado.txt "s3://${BUCKET}/estado.txt"

aws s3api list-object-versions \
  --bucket "$BUCKET" \
  --prefix estado.txt \
  --query 'Versions[].{VersionId:VersionId,IsLatest:IsLatest,LastModified:LastModified,Size:Size}' \
  --output table
```

Validación: aparecen al menos dos VersionId para estado.txt y solo una versión tiene IsLatest=true.

Paso 5 — Consultar una versión anterior

Qué hacemos: Recuperar explícitamente la versión no actual y leer su contenido.

Por qué: Versioning aporta valor cuando podemos volver a observar o recuperar un estado previo después de una modificación.

```
OLD_VERSION=$(aws s3api list-object-versions \
  --bucket "$BUCKET" \
  --prefix estado.txt \
```

```
--query 'Versions[?IsLatest==false][0].VersionId' \
--output text)
```

```
aws s3api get-object \
--bucket "$BUCKET" \
--key estado.txt \
--version-id "$OLD_VERSION" \
estado-v1-recuperado.txt
```

```
cat estado-v1-recuperado.txt
```

Validación: el archivo recuperado muestra estado=CREADO aunque la versión actual del objeto contiene estado=PROCESADO. En consola pueden confirmar lo mismo activando Show versions.

5. Protección y patrón de acceso

Paso 6 — Configurar un lifecycle sencillo

Qué hacemos: Definir una regla que limite la retención de versiones no actuales.

Por qué: Versioning protege frente a sobrescrituras, pero cada versión conserva datos y genera consumo. Lifecycle convierte una intención de retención en una política automática.

17. En el bucket, abran Management > Lifecycle rules y creen dcl-s03-noncurrent-cleanup.
18. Scope: Apply to all objects in the bucket.
19. Acción: Permanently delete noncurrent versions of objects después de 7 días.
20. Activen Delete expired object delete markers. No configuren transiciones a Glacier ni otras clases en este lab.
21. Guarden la regla y registren una captura. No esperen a que ejecute: la evidencia de esta sesión es la configuración activa, no una expiración real.

Validación: la regla aparece Enabled y describe la eliminación de noncurrent versions después de 7 días.

Evidencia mínima de S3

- Bucket con Versioning = Enabled.
- Lista de versiones mostrando dos VersionId de estado.txt.
- Contenido de la versión actual y contenido recuperado de la versión anterior.
- Lifecycle rule dcl-s03-noncurrent-cleanup en estado Enabled.

Contraste arquitectónico — object / block / file

Patrón	Cómo se accede	Qué demuestra S03	Trade-off principal
S3 · object	API/HTTP + bucket/key	Versiones independientes del cómputo	No ofrece semántica de filesystem POSIX
EBS · block	Dispositivo de bloques adjunto a EC2	Contraste conceptual; no hay volumen extra	Es zonal y normalmente se consume como storage de una instancia
EFS · file	NFS montado por varios clientes	Mismo archivo visible desde 2 EC2	Depende de red/mount targets y tiene costo de file service compartido

OJO TÉCNICO: las dos EC2 temporales tendrán un root volume EBS porque el sistema operativo lo necesita. Eso no convierte S03 en un ejercicio de EBS: no creen volúmenes adicionales, no hagan attach/detach y mantengan Delete on termination habilitado.

6. EFS hands-on — el archivo no pertenece a una instancia

Paso 7 — Lanzar dos clientes EC2 temporales

Qué hacemos: Crear una instancia pequeña por AZ para acceder al mismo filesystem desde dos failure domains distintos.

Por qué: La prueba necesita dos clientes independientes. Si el dato solo existiera en el disco local de una instancia, el segundo cliente no podría observarlo como archivo compartido.

22. Lancen dcl-dev-s03-client-a en dcl-dev-subnet-a y dcl-dev-s03-client-b en dcl-dev-subnet-b.
23. AMI: Amazon Linux 2023 estándar; instance type: t3.micro; Proceed without a key pair. Habiliten public IPv4 y asignen dcl-dev-sg-ec2-s03.
24. No agreguen IAM role, user data ni almacenamiento adicional. Confirмен que el root volume usa Delete on termination = Yes.
25. Esperen Instance state = Running y Status checks = 2/2. Conecten a cada instancia desde EC2 > Connect > EC2 Instance Connect > Connect using a Public IP.

Validación: existen dos EC2 running, una por subnet/AZ, ambas accesibles por EC2 Instance Connect y sin key pair ni access keys.

Paso 8 — Montar EFS en el cliente A y escribir

Qué hacemos: Instalar el cliente NFS, montar usando la IP del mount target de la AZ A y crear un archivo dentro del filesystem.

Por qué: Al usar la IP del mount target evitamos convertir la configuración DNS de la VPC en una dependencia del lab. El archivo queda en EFS, no en el root volume de la instancia.

```
sudo dnf install -y nfs-utils
sudo mkdir -p /mnt/dcl

export MT_A_IP="<IP-mount-target-A>"
sudo mount -t nfs \
-o nfsvers=4.1,rsize=1048576,wsz=1048576,hard,timeo=600,retrans=2,noresvport \
"${MT_A_IP}:/mnt/dcl

printf 'writer=%s | created=%s\n' "$(hostname)" "$(date -Iseconds)" \
| sudo tee /mnt/dcl/estado-compartido.txt

sudo cat /mnt/dcl/estado-compartido.txt
mountpoint /mnt/dcl
```

Validación: mountpoint confirma /mnt/dcl y estado-compartido.txt muestra el hostname del cliente A.

7. Prueba compartida — write A → read B → evidencia A

Paso 9 — Montar EFS en el cliente B y leer el mismo archivo

Qué hacemos: Montar el mismo EFS desde la segunda AZ usando la IP del mount target B y leer el archivo creado por A.

Por qué: Dos instancias con roots independientes observan el mismo namespace de archivos porque el estado está en EFS.

```
sudo dnf install -y nfs-utils
sudo mkdir -p /mnt/dcl

export MT_B_IP=<IP-mount-target-B>
sudo mount -t nfs \
-o nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport \
"${MT_B_IP}:/mnt/dcl

sudo cat /mnt/dcl/estado-compartido.txt

printf 'reader=%s | observed=%s\n' "$(hostname)" "$(date -Iseconds)" \
| sudo tee -a /mnt/dcl/estado-compartido.txt

mountpoint /mnt/dcl
```

Validación: el cliente B lee la línea escrita por A y agrega una segunda línea al mismo archivo.

Paso 10 — Volver al cliente A

Qué hacemos: Leer otra vez el archivo desde la instancia A después de que B lo modificó.

Por qué: La evidencia es más fuerte que una simple captura de dos mounts: A puede observar un cambio escrito por B sin copiar archivos entre instancias.

```
sudo cat /mnt/dcl/estado-compartido.txt
```

Validación: desde A aparecen las dos líneas: writer=<cliente-A> y reader=<cliente-B>. El archivo compartido sobreviviría a terminar uno de los clientes mientras EFS siga existiendo.

Evidencia mínima de EFS

- EFS dcl-dev-efs-shared con dos mount targets en las AZ del baseline.
- Security group de EFS con NFS 2049 únicamente desde dcl-dev-sg-ec2-s03.
- Mount exitoso en las dos instancias.
- Archivo creado en A y leído en B.
- Segunda lectura en A mostrando la línea agregada desde B.

Frontera S03/S04: la red es un requisito del mecanismo de acceso, no el tema central de esta semana. Creamos una ruta temporal, un IGW y dos security groups solo para habilitar el experimento; routing, exposición, segmentación y troubleshooting de tráfico se profundizan en S04.

8. Criterios de aceptación y evidencia

- El baseline S01 permanece: dcl-dev-vpc 10.20.0.0/16 y las dos subnets 10.20.1.0/24 / 10.20.2.0/24 en AZ distintas.
- El bucket S3 tiene Versioning habilitado y dos versiones de la misma key.
- El equipo recupera o consulta explícitamente una versión anterior y demuestra que su contenido difiere de la versión actual.
- Existe una Lifecycle rule Enabled para gestionar noncurrent versions.
- EFS tiene un mount target en cada una de las dos AZ del lab y NFS está restringido al SG de las EC2 temporales.
- Las dos EC2 montan el mismo EFS y la evidencia muestra el mismo archivo visible desde ambos clientes.
- El equipo explica por qué S3, EBS y EFS responden a patrones de acceso distintos y no los trata como servicios intercambiables.
- Cleanup validado: no quedan recursos temporales ni costosos de S03.

Respuesta de cierre

Pregunta	Respuesta del equipo
¿Qué estado de S3 sobrevivió a una sobrescritura aparente?	
¿Qué evidencia demuestra que el archivo de EFS no pertenece al root disk de A?	
¿Cuándo preferirías block en lugar de file u object?	
¿Qué trade-off introduces al conservar versiones o un filesystem compartido?	

Checkpoint hacia Semana 4

El estado ya no depende únicamente de una instancia. Ahora queda una nueva pregunta: si varios componentes necesitan llegar a datos y servicios por red, ¿cómo decidimos qué puede comunicarse, por qué ruta y con qué superficie de exposición?

Conserven las evidencias de S3/EFS fuera de AWS, pero restauren la cuenta al baseline de S01. S04 reutilizará la VPC y las subnets, no los recursos temporales de storage/cómputo de esta sesión.

9. Cost awareness, conservación y cleanup

Recursos con costo durante el lab

Recurso	Qué puede generar costo	Control del lab
S3 + versiones	Storage de cada versión + requests	Objeto pequeño; bucket se elimina al cerrar
EFS Regional	Storage y actividad de lectura/escritura	Archivo mínimo; backup automático deshabilitado; eliminar
2 × EC2 t3.micro	Tiempo de cómputo	Crear tarde; terminar apenas capture evidencia
2 × root EBS	Storage de los roots	Tamaño por defecto; Delete on termination
Public IPv4	Tiempo asignado a las EC2	Solo durante el hands-on; se libera al terminar
Internet Gateway	Sin cargo horario propio; puede existir data transfer	Temporal; eliminar porque no pertenece al baseline

CONSERVAR COMO BASELINE

- VPC dcl-dev-vpc · 10.20.0.0/16.
- dcl-dev-subnet-a · 10.20.1.0/24 y su Availability Zone.
- dcl-dev-subnet-b · 10.20.2.0/24 y su Availability Zone.
- Naming/tags del proyecto y las evidencias/checkpoint almacenadas fuera de los recursos temporales de AWS.

ELIMINAR — cleanup obligatorio

26. Desmonten EFS en ambos clientes si las sesiones siguen abiertas: `sudo umount /mnt/dcl`.
27. Terminen dcl-dev-s03-client-a y dcl-dev-s03-client-b. Esperen a que estén Terminated y verifiquen que sus root volumes se eliminaron.
28. En EFS > Network, eliminen los dos mount targets y esperen a que desaparezcan. Después eliminen dcl-dev-efs-shared.
29. Eliminen dcl-dev-sg-efs-s03; después eliminen dcl-dev-sg-ec2-s03, una vez que no existan ENI/dependencias.
30. En S3, usen Empty sobre el bucket y confirmen que elimina todas las versiones y delete markers. Después eliminen el bucket y su lifecycle rule junto con él.
31. Desasocien dcl-dev-rt-s03-public de ambas subnets; verifiquen que vuelven a la main route table. Eliminen dcl-dev-rt-s03-public.
32. Desadjunte y eliminen dcl-dev-igw-s03.
33. Eliminen cualquier objeto, security group, instancia, volume, ENI o recurso accidental creado durante troubleshooting y que no pertenezca al baseline S01.
34. Ejecuten una última revisión por tags/nombres y confirmen que solo permanece lo declarado en CONSERVAR COMO BASELINE.

Validación de cleanup: no existen bucket S03, EFS/mount targets, instancias, root volumes huérfanos, SG S03, route table S03 ni IGW S03. Permanecen únicamente la VPC, las dos subnets y el checkpoint externo.

10. Cierre técnico

- S3 desacopla el objeto del servidor: la key puede mantener historia mediante versiones y una lifecycle policy controla retención.
- EFS entrega semántica de archivos compartidos: dos clientes distintos observan el mismo namespace mediante NFS.
- EBS entrega bloques, no un filesystem compartido administrado. Puede persistir independientemente de una instancia según su configuración, pero sigue siendo un recurso zonal y con un patrón de acceso distinto.
- Persistencia no significa ausencia de trade-offs: conservar más versiones, compartir archivos o mantener capacidad de storage también introduce costo, dependencias y decisiones operativas.

Fuentes oficiales

- [AWS — Retaining multiple versions of objects with S3 Versioning](#)
- [AWS — How S3 Versioning works](#)
- [AWS — Restoring previous versions](#)
- [AWS — Setting an S3 Lifecycle configuration](#)
- [AWS — Creating mount targets](#)
- [AWS — Using VPC security groups with EFS](#)
- [AWS — Mounting EFS with a mount target IP](#)
- [AWS — EC2 Instance Connect prerequisites](#)
- [AWS — Internet gateways](#)
- [AWS — EFS pricing](#)

Decisión de frontera: EFS se monta por IP de mount target para no introducir cambios persistentes en los atributos DNS de la VPC. La conectividad pública temporal existe solo para instalar nfs-utils y usar EC2 Instance Connect; se elimina completa al final. Así S03 demuestra el patrón file sin adelantar la profundidad de networking de S04.